ECE 2544: Fundamentals of Digital Systems
Learning Experience C.1.b: Design and Implementation of Multiple Functional Blocks of a Function Unit

Read the entire specification before you begin working on this project!
Pay particular attention to the rom.txt requirements.

0. Preliminaries

Honor Code Requirements

You must comply with all provisions of Virginia Tech's Honor System. Your Verilog code and submitted documentation must be your own work. Consider all information that you derive from your design process to be proprietary. Your design, coding, debugging, and testing must all derive from your own effort. It is an Honor Code Violation to share *any element of your design or implementation* with *any other person*, or to receive *any element of your work* from *any other person's design* – either from paper, from a computer file, or from the Internet. Review the AI Usage policy in the syllabus.

You may discuss general features of Quartus and the DE10-Lite board with other students. Direct all other questions about your design or implementation to 2544 course staff (instructor, GTA or ULA). Ask your instructor if you have any questions about what is and is not allowed under the Honor Code.

Objectives

After completing this assignment, you should be able to design, simulate, and implement the functional blocks that comprise a function unit from a specification.

Learning Experience C Description

A function unit is a combinational circuit that performs a variety of arithmetic, logic, and shift operations. ***Overall,*** in Learning Experience C (LE C.1.a, LE C.1.b, LE C.2), you will design a function unit that accepts two 8-bit 2s complement operands (OpA, OpB) and a 4-bit function select code (FS) as inputs and performs 13 different operations. The function unit will output the 8-bit result of the operation as well as four status bits (V, C, N, Z).

The function unit is comprised of multiple functional blocks. ***In this assignment (LE C.1.b), you will divide a set of operations into functional blocks, then design and implement those blocks***. Four of those operations must be OR (A ∨ B) or NOR ((A ∨ B)'), NOTA (A'), REM16, and MOD8. You may choose the remaining three operations from the list provided in this specification.

Preparation

Read this project specification completely before beginning any design or Verilog code.
You must have access to a computer that can run the Quartus Prime Lite Edition software package. You will also need the DE10-Lite Board.

- Consult the appropriate sections of Chapter 3 and Chapter 8 of the textbook.
- Review the ECE2544 Lab Manual sections
  - "Switches, Buttons, and LEDs",
  - "Restoring a Quartus Archive",
  - "Simulating in Quartus and Qsim",
  - "Troubleshooting Simulations",
  - "Implementing Circuits on the DE10-Lite Board"
  - "Creating a Quartus Archive for Submission", and
  - "Verilog File Expectations".
- Consult Section 3 of the DE10-Lite User Manual for information on using the switches and LEDs.

- If you have any questions on the use of the DE10-Lite, see a member of the 2544 course staff.

1. Getting Started

1.1 Selecting the Operation Set

You will design and implement functional blocks that can perform seven operations on two 8-bit 2s complement operands, OpA and OpB. You must implement four operations in the Required Operations table, plus three operations selected from the Elective Operations table below. All blocks should generate an 8-bit result.

Table 1.1 Four Required Operations.

	Operation name	Output	Description
EITHER	or	A ∨ B	Bitwise OR of A and B
	nor	(A ∨ B)'	Bitwise NOR of A and B
REQUIRED	notA	A'	1s complement of A
	rem16	B rem 16	Remainder of B by 16 (assume a truncation function, such that the quotient is rounded toward zero and the remainder has the same sign as B)
	mod8	B mod 8	Modulus of B by 8 (assume a floor function, such that the quotient is rounded toward negative infinity and the remainder is positive)

Table 1.2 Elective Operations. (Choose 3.)

Operation name	Output	Description
and	A ∧ B	Bitwise AND of A and B
nand	(A ∧ B)'	Bitwise NAND of A and B
xor	A ⊕ B	Bitwise XOR of A and B
xnor	(A ⊕ B)'	Bitwise XNOR of A and B
notB	B'	1s complement of B
lslB (logical shift left)	B << 1	Shift B left by 1 bit; the vacant bit positions are filled in with 0
lsrB (logical shift right)	B >> 1	Shift B right by 1 bit; the vacant bit positions are filled in with 0
cslB (circular shift left)	B << 1	Shift B left by 1 bit; the vacant bit positions are filled in with the bits that are shifted out of the sequence
csrB (circular shift right)	B >> 1	Shift B right by 1 bit; the vacant bit positions are filled in with the bits that are shifted out of the sequence
asrA (arithmetic shift right)	A >> 1	Shift A right by 1 bit; the MSB position does not change

You will need to divide the seven operations selected into functional blocks, then design and implement each block. ***You must use between two and four functional blocks.*** As part of your submission, you will need to include an explanation of how and why you divided the operations into blocks.

***Tip**: Notice that Table 1.1 tells you to use a truncation function for the remainder operation and a floor function for the modulus operation. This results in a difference between modulus and remainder for negative operands, i.e. -7 rem 3 = -1 while -7 mod 3 = 2. Why is there a difference? Both remainder and modulus can be said to be equal to*

Dividend – Divisor × Quotient

The difference lies in the value of the Quotient.

For mod, we use a floor function to find the quotient (closer to negative infinity).
For rem, we use a truncation function to find the quotient (closer to zero).
(For positive numbers, these are both the same.)

Scientists and engineers could debate for hours about the "true" meaning of remainder or modulus. The explanation above will be used for ECE 2544 without debate.

1.2 Navigating the Quartus Archive

1. Download the LE C.1.b Quartus archive and create a working folder for Learning Experience C.1.b (e.g. c:\ece2544\LE_C1b)

***Tip**: All your Quartus files for Learning Experience C.1.b, including waveform/simulation files, must be stored in this working folder (not in sub folders).*

2. Follow the directions in the lab manual to restore the archive **LE_C1b_starter.qar**. When prompted, select the working folder you just created as the destination folder.

***Caution**: Once you have restored an archive, use the appropriate project file (*.qpf) to open the project in future instances. If you open the qar archive after restoring it for the first time, you could overwrite your saved work.*

3. Open **LE_C1b.bdf**.

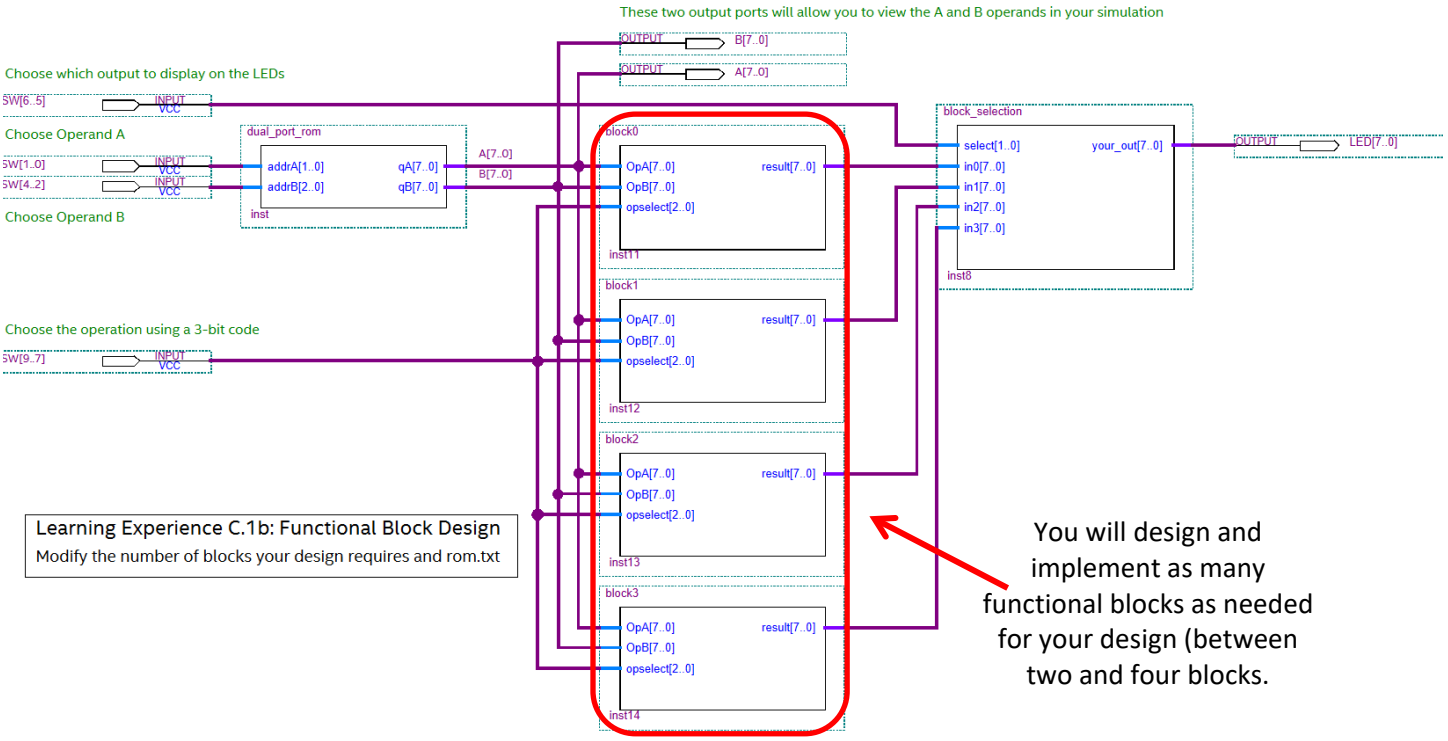


Figure 1.1. Schematic of full system for Learning Experience C.1.b. Your responsibility is to implement the "block#" modules. Only edit the number of blocks that you need. For example, if your design includes two functional blocks, you will edit block0 and block1.

LE_C1b.bdf shows the top-level design. It contains three different components: a ROM, which contains a set of 8-bit operands, functional blocks labeled ***block0, block1, block2*** and ***block3,*** and a component that can choose which block output will be displayed on the LEDs. You will implement each of your functional blocks in the block# blocks. The rest of the system has been provided for you.

Right-click ***block0*** and choose ***Open Design File*** to open the Verilog model. Another way to do this is to choose Files in the Project Navigator, then double-click *block0.v*. The existing *block0.v* file contains a template you will fill in with your circuit design. All of the functional block circuits take as input the two 8-bit operands A and B, and three of the switches on the DE10-Lite board (SW[9:7]).The switches control which operation your block will output at any one time. This means that as part of your design you will have to assign a 3-bit code to each operation. Edit the number of blocks required for your design. If you have divided your operations into two functional blocks, implement them in block0 and block1. If you have divided your operations into three functional blocks, implement them in block0, block1, and block2.

The operands A and B will be supplied by a small Read Only Memory (ROM). The ROM contains eight 8-bit operands, which can be selected using the other switches on your DE10-Lite board. SW[1:0] select operand A and SW[4:2] select operand B, as described below.

4. Choose ***Files*** in the Project Navigator, then double-click ***rom.txt***.

Each line of the text file has the hex value for one memory location. Since each word is 8 bits wide, each line of the file has a 2-digit hex number. For example, the 8-bit value in memory location 0 is 0x0F, or 0b00001111. You can select operand A from the first four of these values using SW[1:0]. You can select operand B from all eight values using SW[4:2]. For example, to select the contents of memory location 2 as operand A, set SW[1:0] to 10; to select the contents of memory location 6 as operand B, set SW[4:2] to 110.

You can modify the contents of the ROM by editing rom.txt.

> ***Tip****: This is slightly different from LE C.1.a because you can only choose the first four data values in the rom for operand A (locations 0-3). You can still choose any of the eight data values in the rom for operand B.*
>
> ***Tip****: Just like LE C.1.a, this means operand A and operand B must be selected from data contained in the rom. You cannot input them directly, and they should not be assigned input values in simulations. To choose a specific operand data value, save it in the rom and use the switches to select it.*

2. Circuit Design

Once you have selected your operation set, you must:

- Divide the operations into groups of operations that can be implemented in a similar way. You must have between two and four groups, which will result in between two and four functional blocks.
- For each functional block, design a circuit that accepts 8-bit operands and correctly outputs the result of all operations in the group. Be sure to consider efficient use of transistor resources in your design.
- Assign a unique 3-bit code to each operation that will be used to select that operation using SW[9:7].
- Create and simulate a *Verilog model* in Quartus for each functional block.
- Implement and test your circuit design on your DE10-Lite Board.
- Prepare your files for submission.

> ***Tip****: Design each functional block separately. Each block# module should operate independently of the others.*

2.1 Implementation

The Quartus archive project file posted for this project includes the schematic and Verilog files necessary to build the system. Do not modify any files except for the `rom.txt` file and the `block#.v` files. Only modify the number of block files required for your design. In particular, if you change the top-level schematic or modify the pin assignments, there is a chance you could damage your DE10-Lite board.

The port declaration for each block# module is as follows:

```
module block# (result, OpA, OpB, opselect);
input [2:0] opselect;
input [7:0] OpA, OpB;
output [7:0] result;
```

As shown on the schematic, the inputs `OpA` and `OpB`, are supplied from the ROM output. The opselect input is connected to the DIP switches, SW[9:7], and is used to select the operation. The 8-bit output, `result`, is connected to the block selection component, which will allow you to choose which block output is displayed on the LEDs using switches SW[6:5]. As always, LED[7] is the most significant bit. Setting SW[6:5] to 00 will display the output from *block0.v*; setting SW[6:5] to 01 will display the output from *block1.v*; setting SW[6:5] to 10 will display the output from *block2.v*; setting SW[6:5] to 11 will display the output from *block3.v.*

As part of your design process, you will have to assign unique 3-bit code values for each operation. Do not simply assign an arbitrary operation code to a particular operation: You will find that groups of operations have similar structures such that a careful choice of the codes will simplify the logic for controlling the operations. As in LE C.1.a, you can choose which operation result is output by setting SW[9:7] to the corresponding 3-bit code.

> ***Tip**: If any of your functional blocks require less than three control bits, you still need to assign a unique 3-bit code to each operation. If one or more of the bits is irrelevant, then you should easily be able to assign values to them that will provide unique codes for each operation.*

2.2 Requirements and constraints

1. You are permitted to modify only the Verilog files *block0.v*, *block1.v, block2.v and block3.v,* the ROM contents file (*rom.txt*) and waveform files. You must not modify any other file or schematic in the project. You must not add or remove any files, other than waveform files, to the project. Any additional modules that you might need to create to implement each functional block should be included in the corresponding *block#.v* file.
2. You are not permitted to modify the module or port declarations of the *block0*, *block1, block2 and block3* modules.
3. Port declarations must use the ordered list port connection method. Dotted notation is not permitted.
4. You must implement your design using structural and dataflow Verilog constructs (gate primitives, assign statements with operators). ***You may not use dataflow arithmetic operators*** (e.g. assign F = A + B; or assign F = A – B)., ***or any other operators listed on the "Other Dataflow Operators" slides.***
5. You are not permitted to create any schematics or to use behavioral Verilog constructs (e.g., case statements, for loops, always, generate, etc).
6. Your design must be completely combinational.
7. You must assign a unique 3-bit code to each operation.
8. Your *rom.txt* file must include the last four digits of your student ID number and the validation data described in the Completion section below.

2.3 Simulation

Verify the behavior of each functional block by simulating it in Quartus. Be sure to test multiple operand values for each operation. You should carefully consider which sets of inputs are used to test your function unit so that a wide variety of cases are covered. For example, you must verify operations using all combinations of positive and negative values. For single-operand operations, this will require at least two; for two-operand operations at least four.

> ***Tip**: While simulating individual blocks, be sure to set the SW[6:5] signals to select the block you are testing. If you do not assign input values to SW[6:5], you will have no output.*
>
> ***Tip**: You may find it helpful to ungroup SW and then group meaningful bits together, e.g. SW[9:7], SW[6:5], SW[4:2], SW[1:0].*

Once you verify the behavior of each functional block individually, simulate the entire system using SW[6:5] to choose the functional block displayed, SW[9:7] to select the operation, and SW[4:2] and SW[1:0] to choose the operands. Be sure to test multiple operand values for each operation. Your output signals should be LED[7:0], and if you would like to see the selected operand values, A[7:0] and B[7:0]. *Note that A and B are **output** signals, **not input** signals. Do not assign values to A and B in your input waveform.* You will need to include your simulations in your submission. See the Completion section below.

> ***Tip**: A and B are **not** input ports; they are the output of the rom. You should not be assigning 8-bit values to A and B in your input waveform. You must select operand A and operand B using SW[1:0] and SW[4:2], respectively. If you would like to see operand values in your simulation, you can add the output of the rom module using nodes A and B, as shown in the Learning Experience C.1.a specification. Do **not** assign values to these signals in your input waveform. They are **outputs**.*

Once your waveform displays correct behavior, screenshot the waveform to be inserted in your LE documentation (see the Completion section). Be sure the signal names and the input values are readable; otherwise the waveform is meaningless.

2.4 Testing

After your model simulates satisfactorily, you must compile it and then program the DE10-Lite board with it. Please refer to the lab manual for full instructions. Briefly, to compile your design, double-click **Compile Design** in the Tasks window, or select **Processing**->**Start Compilation** from the toolbar. When the project is successfully compiled, connect the board to your computer using the USB cable provided with the board. Then double-click on **Program Device (Open Programmer)** in the Tasks window, or select **Tools**->**Programmer** from the toolbar. The programmer window should open. Click on the **Start** button. The start button will gray out, and the LEDs on the DE10-Lite board will dim. In a few seconds, the Progress indicator on the programmer window should turn green and say 100% (Successful). Your design has now been downloaded to the board.

> ***Tip**: If you make changes to the operands in the rom.txt file, you must recompile in order to see the changes.*

Test your circuit by using the switches on the DE10-Lite board to provide valid input combinations to the functional blocks that you implemented on the FPGA. A switch provides logic-1 when it is in the up position and logic-0 when it is in the down position. As shown in Figure 2.1, use SW[6:5] to choose which functional block output will be displayed on the LEDs. Use SW[9:7] to apply each 3-bit code that you assigned to each operation. Use SW[1:0] to choose the A operand from the first four operands contained in rom.txt. Use SW[4:2] to choose the B operand from all eight operands contained in rom.txt.

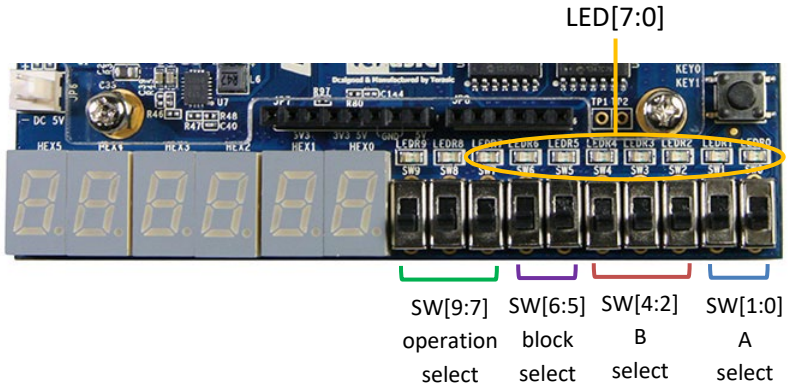


Figure 2.1. DE10-Lite board switches used for input and LEDs used for output.

> ***Tip**: When you do a full compile, you will see a warning that informs you there is no exact pin location assignment(s) for 16 pins of 34 total pins. The A[7:0] and B[7:0] output ports were added to the project so that you would be able to view the operand values when you simulate. Because these were added to facilitate simulation, they are not actually connected to the I/O on the board. This is fine.*

2.5 Preview: Cost and Speed Calculations

You are not required to submit cost and speed calculations with this assignment (transistor count and propagation delay), but you are expected to consider efficient use of transistor resources in your design. However, you *will* be using the functional blocks designed here to build your entire function unit in Learning Experience C.2, and you will be required to submit cost and speed calculations with LE C.2, as well as an explanation of your calculations and any assumptions. Determining transistor count and propagation delay is much easier if you do not wait until you complete your entire design. You should keep track of the transistor count and propagation delay as you go along, for each subcircuit in your design. To prepare for LE C.2, you can determine the number of transistors utilized and the propagation delay of your functional blocks now. You may use the primitive gate propagation delays assumed in previous assignments, or treat them as variables at this point.

> ***Tip**: Organizing your transistor count and propagation delay information into a spreadsheet now will save you a lot of time later. When cost and speed calculations are required for Learning Experiences, it is not sufficient to simply provide a number. It must be clear how you arrived at that number. You must understand which gates you used in your circuit, and how CMOS transistors are used to build each of those gates. If your tables are well-organized, you will be able to easily modify them when you add on to your circuit, instead of starting over for every LE.*
>
> ***Tip**: If you implemented your circuit in Verilog using ANDs and ORs, you cannot assume it is implemented using NANDs. That is what "as implemented" means.*
>
> ***Recall:** CMOS means complementary pairs of n-type (NMOS) and p-type (PMOS) MOSFETs. All circuits in this assignment and course must be implemented using CMOS processes. Disregarding this requirement will result in deductions. You must understand how logic gates are implemented in CMOS in order to correctly determine your transistor count.*

3. Completion

3.1 Documentation file

This is a minor Learning Experience. The documentation requirements for this assignment are minimal. You should submit a single pdf or doc/x file containing the following items in the order listed below.

- An explanation of the manner in which you divided the operations into functional blocks, alternatives considered, and the reasons you made the final design decisions. Please note that this section will comprise a nontrivial portion of your grade for this assignment.
- Simulation waveforms demonstrating correct operation of your functional blocks for multiple operand values for each operation. You should carefully consider which sets of inputs provide a reasonable test of the functionality of each operation. See Section 2.3.

3.2 Modify the rom.txt file for validation

Once you are confident that your design is working on the DE10-Lite board, you must edit the `rom.txt` file to contain the data shown in Table 3.1 before creating your project archive. Review the explanation of rom.txt in section 1.2. Remember that each 8-bit value stored in the rom must be expressed in hex in rom.txt.

Memory locations 0 and 1 must contain the BCD code for the last four digits of your VT ID number. Four digits will require four 4-bit codes, for a total of 16 bits. Each location in the rom holds eight bits. The first eight bits should go in memory location 0; the second eight bits in memory location 1. The remaining six locations should contain the values listed in Table 3.1 below.

Table 3.1 ROM contents of addresses 0-7 to be used for validation.

ROM address	Contents in binary 0b	Contents in hex 0x	How rom.txt should look using the example ID 1234	How rom.txt should look using the example ID 9876
0	1st 8 bits of the BCD code for the last four digits of your ID	Depends on ID	@0 12 34 75 9E EE D3 D5 58	@0 98 76 75 9E EE D3 D5 58
1	2nd 8 bits of the BCD code for the last four digits of your ID	Depends on ID		
2	01110101	75		
3	10011110	9E		
4	11101110	EE		
5	11010011	D3		
6	11010101	D5		
7	01011000	58		

***Tip**: Editing all 8 locations in rom.txt is **required** for submission and validation. Do not skip it.*

***Tip**: Repeat the examples below using your ID number. If you have questions about BCD encoding of your ID number, see a member of the 2544 course team **before** you submit.*

Example 3.1: If your ID number is 905551234, the last four digits are 1234. The BCD code that represents 1234 is 0001001000110100. The first eight bits, 0b00010010 (0x12), should go in location 0; the second eight bits, 0b00110100 (0x34) should go in location 1.

Example 3.2: If your ID number is 905559876, the last four digits are 9876. The BCD code that represents 9876 is 1001100001110110. The first eight bits, 0b10011000 (0x98), should go in location 0; the second eight bits, 0b01110110 (0x76), should go in location 1.

The contents of rom.txt is expressed in hex; therefore, for these two examples, rom.txt should contain the values shown in the last two columns of Table 3.1 on the previous page.

3.3 Circuit Validation

Following submission, your circuit will be validated by 2544 course staff. You will provide the source files that will allow us to compile the same files that you used to implement your functional blocks on the DE10-Lite Board. ***After*** you complete the implementation ***and edit rom.txt*** according to Table 3.1, archive your solution and save it as **LE_C1b_YOURPID.qar**. (Replace YOURPID with your pid, just as you did in previous assignments.)

Instructions for creating a Quartus archive (*.qar file) for submission can be found in the lab manual. When you create the archive, you can choose the filename and where it will be stored. Make sure you name the qar file appropriately; you do not want to confuse it with the original qar file provided with the project. Upload the qar file to Canvas with your submission, ***making certain that you upload the completed archive that contains your work, and not the one that was provided to you***. The file that you submit is the file that will be graded.

> ***Tip****: Be sure to choose a filename that will allow you to differentiate between your new archive and the one that was provided to you. (**See the lab manual and the help video on the ECE 2544 website under Supplemental Material: Hardware and Software.**)*
>
> ***Tip****: You must edit rom.txt to contain the required validation data before creating the archive. Otherwise, your archive will not be correct.*

Complete the validation sheet. You will need to enter the last four digits of your ID, and indicate which operations you implemented in which block and the 3-bit code assigned to each operation. Complete the table for each block that you used. Only use the number of rows required. For example, if you implemented four operations in block 0, fill in four rows of that table.

> ***Tip****: We will validate your circuit after you submit. It is good engineering practice to verify that the validation procedure, as shown on the validation sheet, produces correct results, by testing it on your board before you submit it. Your validation results should not be a surprise.*
>
> ***Tip****: If you do not complete the required information on the validation sheet, we will not be able to validate your circuit and your validation score will be zero.*

3.4 Submission Requirements

Submit the following files on Canvas. **Do not put them in a zip file!**

1. Your Archived Project **LE_C1b_*YourPID*.QAR.** *This must contain the correct submission rom.txt* (See Table 3.1).
2. Your completed Verilog files **block0.v, block1.v, block2.v** (if needed)**, block3.v** (if needed)**.** *These must match the .v files in your archive.*
3. Your modified memory file **rom.txt** *This must match the rom.txt file in your archive.* (See Table 3.1.)
4. Required documentation in a single pdf or doc/x file (see above).

5. Your validation sheet. Remember to complete the tables listing your operation set, the block in which each operation is implemented, and the 3-bit code assigned to each operation.

> ***Tip**: You are required submit your qar <u>and</u> your .v and rom.txt file. The purpose for this requirement is to facilitate efficient grading. Failure to submit any of the required files will result in point deductions.*
>
> ***Tip**: **Your PID is the <u>text</u> of your VT email address before "@vt.edu"**. It is <u>not</u> your ID number, which starts with "90…".*

3.5 Grading

The design project will be graded on a 100 point basis, as shown in the Canvas Assignment rubric.